SIMULATION EXPERIMENT – 3

I2C-BASED HEALTH MONITORING NODE FOR PULSE AND TEMPERATURE ACQUISITION

➔ PROGRAM:

```
#include <Wire.h>


#define LM75_ADDRESS 0x48

#define PULSE_PIN 8


unsigned long pulseCount = 0;

unsigned long lastTempTime = 0;

unsigned long lastBpmTime = 0;


int previousPulseState = LOW;


float readTemperature()

{

 Wire.beginTransmission(LM75_ADDRESS);

 Wire.write(0x00);

 Wire.endTransmission();


 Wire.requestFrom(LM75_ADDRESS, 2);


 if (Wire.available() >= 2)

 {

  byte msb = Wire.read();

  byte lsb = Wire.read();
```

```
   int16_t rawTemperature =
     ((int16_t)msb << 8) | lsb;

   return rawTemperature / 256.0;
  }

  return -999.0;
}

void setup()
{
  Serial.begin(9600);
  Wire.begin();

  pinMode(PULSE_PIN, INPUT);

  Serial.println("I2C HEALTH MONITORING NODE");
  Serial.println("--------------------------");
}

void loop()
{
  unsigned long currentTime = millis();

  /* Pulse counting */

  int currentPulseState = digitalRead(PULSE_PIN);
```

```
if (currentPulseState == HIGH &&
   previousPulseState == LOW)
{
  pulseCount++;
}

previousPulseState = currentPulseState;

/* Temperature acquisition every 1 second */

if (currentTime - lastTempTime >= 1000)
{
  lastTempTime = currentTime;

  float temperature = readTemperature();

  Serial.print("Temperature: ");

  if (temperature > -100)
  {
    Serial.print(temperature, 1);
    Serial.println(" C");
  }
  else
  {
    Serial.println("Sensor Error");
  }
}
```

```
 /* BPM calculation every 5 seconds */


 if (currentTime - lastBpmTime >= 5000)
 {
   lastBpmTime = currentTime;

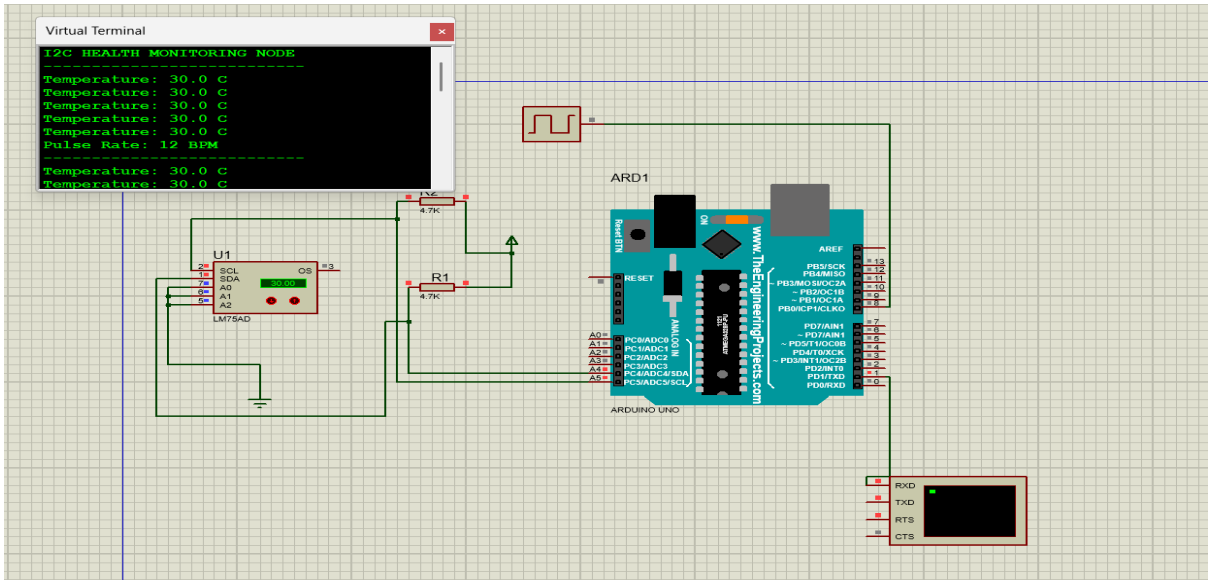

   float bpm = (pulseCount * 60.0) / 5.0;


   Serial.print("Pulse Rate: ");
   Serial.print(bpm, 0);
   Serial.println(" BPM");


   Serial.println("---------------------------");


   pulseCount = 0;
 }
}
```

Output



Reflection

Through this experiment, I learned how to interface the LM75AD temperature sensor with the Arduino UNO using I²C communication. I understood the importance of SDA and SCL lines, pull-up resistors, and the Wire library for acquiring temperature data. I also learned how to generate pulse signals using the CLOCK source, count pulses through digital pin D8, and calculate the pulse rate in BPM. This experiment helped me understand how temperature and pulse-rate data can be processed and displayed through the Proteus Virtual Terminal. Overall, I gained practical knowledge of embedded system design and health-monitoring applications.